## Fachinformatiker/-in Fachrichtung Systemintegration **FA 235**

## Analyse und Entwicklung von Netzwerken

**Lösungsvorschläge:** **Lösungsvorschläge sind in der Regel Vorschläge der einreichenden Schulen; sie sind im Wortlaut nicht bindend. Anderslautende, aber zutreffende Antworten sind ebenfalls als richtig zu werten.**

**Nur für die Hand des Prüfers! Punkte**

**Aufgabe 1 Netzwerk planen** **30**

1.1.1 z. B.: 2
- kleinere Broadcast-Domains
- bessere Störsicherheit
- leichtere Implementierung von Sicherheitszonen
- bessere Abbildung von Ordnungsstrukturen (z.B. Abteilungen, Anwendungen)

1.1.2 Core-Layer: 6
Schnelles Routing zwischen den Distribution-Switches. Üblicherweise keine Sicherheitsfunktionen.

Distribution-Layer:
Verteilerschicht zwischen Core- und Access-Layer. InterVLAN-Routing und Sicherheitsfunktionen (ACLs).

Access-Layer:
Anbindung der Endgeräte.

1.2 12

| Netz | Teilnehmer | Netzgröße | Netzadresse | Broadcast-adresse |
|---|---|---|---|---|
| 1 | 310 | 512 | 10.30.0.0/23 | 10.30.1.255 |
| 2 | 180 | 256 | 10.30.2.0/24 | 10.30.2.255 |
| 3 | 122 | 128 | 10.30.3.0/25 | 10.30.3.127 |
| 4 | 78 | 128 | 10.30.3.128/25 | 10.30.3.255 |
| 5 | 45 | 64 | 10.30.4.0/26 | 10.30.4.63 |
| 6 | 31 | 64 | 10.30.4.64/26 | 10.30.4.127 |
| 7 | 12 | 16 | 10.30.4.128/28 | 10.30.4.143 |

1.3.1 Campus-Netzwerk: 2001:db8:ea71:2a00::/57 6
Rechenzentrum: 2001:db8:ea71:2a80::/58
Reserve: 2001:db8:ea71:2ac0::/58

1.3.2 /64 - /58 = 6 Bit → $2^6$ = 64 Subnetze 1

1.4.1 Verfügbarkeit 1

1.4.2 z. B.: 2
- paarweiser Einsatz von Core- und Distribution-Switches.
- paarweiser Einsatz von Links (LAGs).
- First-Hop-Redundanz (z. B. VRRP, HSRP).
- Switches mit 2 Netzteilen

**Aufgabe 2 Fehleranalyse durchführen** **10**

2.1 Ziel: 0.0.0.0/0
Next-Hop: 10.0.8.1 3

alternativ: **ip route 0.0.0.0 0.0.0.0 10.0.8.1**

2.2 Das in der Routingtabelle vorhandene Netzwerk 10.8.36.0/23 (VLAN 13) ist nicht mehr in dem vom Core-Switch gerouteten Summary-Netzwerk 10.8.32.0/22 enthalten und damit von dort aus nicht erreichbar. 4

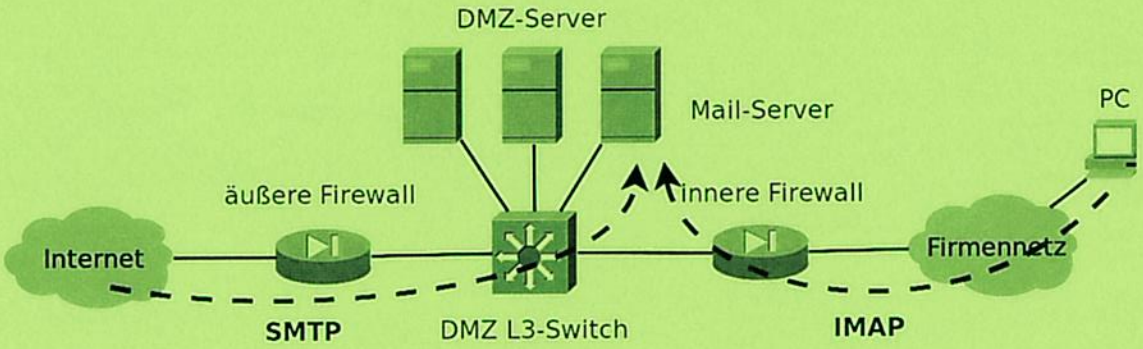
2.3 Eine Alternative zum statischen Routing ist das dynamische Routing.
Hierbei wird die Default-Route mit Hilfe eines Routing-Protokolls (z. B. OSPF) erlernt. 3

**Aufgabe 3 IT-Sicherheit planen** **18**

3.1.1 Stateless Firewalls (z. B. ACLs) treffen die Entscheidung (permit/deny) für jedes einzelne Paket neu. Stateful Firewalls merken sich vorherige Entscheidungen (Connection Tracking Table) und können diese bei nachfolgenden Entscheidungen einbeziehen. 3

3.1.2 Bei Deep Packet Inspection wird der Inhalt von Paketen auf Layer 7 überprüft. Die Firewall muss dazu die entsprechenden L7-Protokolle kennen (z. B. HTTP). 2

3.2.1 z. B.: 3




3.2.2 siehe Lösung zu 3.2.1 4

3.2.3 z. B.: 2
- Web-Server
- Web-Proxy
- VPN-Server
- ...

3.3.1 Die Reihenfolge der Regeln stimmt nicht. Die untere Deny-Regel wird nie ausgeführt, da vorher immer die Permit-Any-Regel ausgeführt wird. Die Permit-Any-Regel muss am Ende der ACL stehen. 2

3.3.2 Eine Permit-Any-Regel passt auf alle Quell-Adressen und damit auch die öffentlichen Quell-Adressen im Internet. Soll eine ACL direkten Zugang (ohne Proxy) zum Internet gewähren, muss am Ende der ACL eine permit any Regel stehen. 2

**Aufgabe 4 Switch über REST-API abfragen** 16

4.1 Quellcode: 6

```python
def is_any_link_down(ports):

    # Gehe alle Ports im Dictionary durch
    for port_name, status in ports.items():

        # Überprüfen, ob der Link-Status False ist
        if status["link"] == False:
            return True  # Sofort True zurückgeben, da ein Link down ist

    return False


# Beispiel-Dictionary für Tests
ports = {
    "gi0/1": {"enabled": True, "link": False, "speed": ""},
    "gi0/2": {"enabled": True, "link": True, "speed": "1G"},
    "gi0/3": {"enabled": True, "link": True, "speed": "100M"},
}

# Aufruf der Funktion und Ausgabe des Ergebnisses
print("Ergebnis der Funktion:", is_any_link_down(ports))
```

Pseudocode:

- Definition der Funktion *`is_any_link_down()`* mit dem Parameter *ports*.
- Jeder Schlüssel im Dictionary repräsentiert einen Port (z. B. `"gi0/1"`).
- Schleife über die Ports: Extrahiere den `status` des aktuellen Ports. Der `status` ist ein weiteres Dictionary mit den Schlüsseln `"enabled"`, `"link"` und `"speed"`.
- Prüfe den Wert des `link`-Schlüssels im `status`-Dictionary:
  - Falls `link == False`: Gib `True` zurück. Andernfalls: Setze die Schleife fort.
- Falls die Schleife beendet wird, ohne dass `link == False` gefunden wurde:
  - Gib `False` zurück.

4.2 Quellcode: 8

```python
def num_slow_ports(ports):
    sum = 0
    for port_name, status in ports.items():
        if status["speed"] == "10M" or status["speed"] == "100M":
            sum +=1
    return sum

# Beispiel-Dictionary für Tests
ports = {
    "gi0/1": {"enabled": True, "link": True, "speed": "10M"},
    "gi0/2": {"enabled": True, "link": True, "speed": "1G"},
    "gi0/3": {"enabled": True, "link": True, "speed": "100M"},
    "gi0/4": {"enabled": False, "link": False, "speed": ""},
    "gi0/5": {"enabled": True, "link": True, "speed": "10M"},
}

# Aufruf der Funktion und Ausgabe des Ergebnisses
print(num_slow_ports(ports))  # Erwartete Ausgabe: 3
```

Pseudocode:

- Definition der Funktion `num_slow_ports()` mit dem Parameter `ports`.
- Eine Zählvariable erstellen und auf 0 setzen.
- Schleife über die Ports: Für jedes Schlüssel-Wert-Paar im `ports`-Dictionary: Aufrufen der Geschwindigkeit im `status-Dictionary`
- Überprüfung der Geschwindigkeit: Falls `speed` gleich `"100M"` oder `"10M"`: Erhöhe die Zählvariable um 1.
- Nach der Schleife: Gib den Wert der Zählvariable zurück.

4.3 Die HTTP-Methode **GET** ist in REST-APIs die gängige Wahl für Statusabfragen von Ressourcen. 2

**Aufgabe 5 Kostenkalkulationen** **16**

5.1 8

| Bezeichnung | Stück | Stückpreis | Gesamtkosten |
|---|---|---|---|
| Anschlussdosen | 1000 | 200 € | 200.000 € |
| Switches | 50 | 500 € | 25.000 € |
| VoIP-Anlage | 1 | 2.000 € | 2.000 € |
| VoIP-Telefone | 450 | 100 € | 45.000 € |
| Fortbildung | | | 6.000 € |
| Gebühren | | | 6.000 € |
| | | | **284.000 €** |

5.2 5

Arbeitszeit 1: 21 Tage / Mitarbeit à 8 Stunden = 168 Stunden x 2 MA = 336 Stunden
Arbeitszeit 2: 21 Tage à 1,5 Stunden = 31,5 Stunden x 2 MA = 63 Stunden
Arbeitszeit 3: 5 Tage à 6 Stunden = 30 Stunden x 1 MA = 30 Stunden
Gesamt: 429 Stunden

429 Stunden x 50 € = 21.450 €

5.3 3

284.000 € + 21.450 € = 305450 € / 20.000 € = 15,27 = 15 Jahre

Oder: 280.000 € + 22.000 € = 302.000 € / 20.000 € = 15,1 = 15 Jahre